

LINUX GREP, REGEX & FILTERING

Lab-4

Name: Gopika Shree

Register No.: 261100690031

Part A – Basic grep and Filtering

Display all lines containing the word failed.

grep 'failed' security_log

```
(gopika@kali)~[/linux/lab/lab4]
$ grep 'failed' security_log
```

Display all lines containing failed, ignoring case.

grep -i 'failed' security_log

```
(gopika@kali)~[/linux/lab/lab4]
$ grep -i 'failed' security_log
[2025-08-18 08:12:03] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:12:10] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:12:17] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:29:58] Failed login for user guest from 10.1.2.3 port 22
[2025-08-18 08:30:17] Failed login for user guest from 10.1.2.3 port 22
[2025-08-18 08:30:44] Failed login for user admin from 172.16.5.10 port 2222
[2025-08-18 08:31:03] Failed login for user admin from 172.31.200.9 port 2222
[2025-08-18 08:42:01] Failed login for user root from 198.51.100.77 port 2222
[2025-08-18 08:42:19] Failed login for user root from 198.51.100.77 port 2222
[2025-08-18 08:42:38] Failed login for user admin from 198.51.100.77 port 2222
```

Display all lines that contain either Failed or Denied.

grep -E 'Failed|Denied' security_log

```
(gopika@kali)~[/linux/lab/lab4]
$ grep -E 'Failed|Denied' security_log
[2025-08-18 08:12:03] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:12:10] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:12:17] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:29:58] Failed login for user guest from 10.1.2.3 port 22
[2025-08-18 08:30:17] Failed login for user guest from 10.1.2.3 port 22
[2025-08-18 08:30:44] Failed login for user admin from 172.16.5.10 port 2222
[2025-08-18 08:31:03] Failed login for user admin from 172.31.200.9 port 2222
[2025-08-18 08:42:01] Failed login for user root from 198.51.100.77 port 2222
[2025-08-18 08:42:19] Failed login for user root from 198.51.100.77 port 2222
[2025-08-18 08:42:38] Failed login for user admin from 198.51.100.77 port 2222
```

Display all lines that do not contain the word INFO.

grep -v 'INFO' security_log

```
(gopika@kali)~[/linux/lab/lab4]
$ grep -v 'INFO' security_log
[2025-08-18 08:12:03] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:12:10] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:12:17] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:13:02] WARNING Multiple authentication failures from 203.0.113.45
[2025-08-18 08:15:44] ERROR Connection denied from 198.51.100.77 port 9001
[2025-08-18 08:16:05] Suspicious connection from 198.51.100.77 to evilc2.ddns.net:9001
[2025-08-18 08:16:19] Suspicious outbound request https://evilc2.ddns.net/update.php
[2025-08-18 08:18:07] WARNING Request to http://malware-test.example.org/download.php from 10.20.30.40
[2025-08-18 08:18:20] Suspicious file download evil.exe from http://malware-test.example.org/evil.exe
```

Display all lines containing the exact word root.

grep -w 'root' security_log

```
(gopika@kali)-[~/linux/lab/lab4]
$ grep -w 'root' security_log
[2025-08-18 08:12:03] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:12:10] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:12:17] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:27:24] Suspicious crontab created for user root
[2025-08-18 08:42:01] Failed login for user root from 198.51.100.77 port 2222
[2025-08-18 08:42:19] Failed login for user root from 198.51.100.77 port 2222
```

Display all occurrences of the word attack along with their line numbers.

grep -n 'attack' security_log

```
(gopika@kali)-[~/linux/lab/lab4]
$ grep -n 'attack' security_log
17:[2025-08-18 08:20:10] WARNING Email received from attacker123@phishing-example.com
(gopika@kali)-[~/linux/lab/lab4]
```

Count the number of lines containing ERROR.

grep -c 'ERROR' security_log

```
(gopika@kali)-[~/linux/lab/lab4]
$ grep -c 'ERROR' security_log
10
(gopika@kali)-[~/linux/lab/lab4]
$ grep -l 'malware' *.log
```

Display only the filenames of files whose contents contain the word malware.

grep -l 'malware' *.log

```
(gopika@kali)-[~/linux/lab/lab4]
$ grep -l 'malware' *.log
grep: *.log: No such file or directory
(gopika@kali)-[~/linux/lab/lab4]
$ grep -l 'malware' *.log
security_log
(gopika@kali)-[~/linux/lab/lab4]
```

Display the first five lines containing Suspicious.

grep -i 'Suspicious' security_log | head -n 5

Display the last five failed-login entries.

grep -i 'failed.*login\\|login.*failed' security_log | tail -n 5

```
(gopika@kali)-[~/linux/lab/lab4]
$ grep -i 'failed.*login\\|login.*failed' security_log | tail -n 5
[2025-08-18 08:30:44] Failed login for user admin from 172.16.5.10 port 2222
[2025-08-18 08:31:03] Failed login for user admin from 172.31.200.9 port 2222
[2025-08-18 08:42:01] Failed login for user root from 198.51.100.77 port 2222
[2025-08-18 08:42:19] Failed login for user root from 198.51.100.77 port 2222
[2025-08-18 08:42:38] Failed login for user admin from 198.51.100.77 port 2222
```

Part B – Regular Expression Extraction

Extract all IPv4 addresses from the log.

grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' security_log

```
(gopika@kali)-[~/linux/lab/lab4]
$ grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' security_log | head -5
192.168.10.5
192.168.10.21
203.0.113.45
203.0.113.45
203.0.113.45
```

Extract all IP addresses belonging to the 10.x.x.x private network.

`grep -Eo '\b10(\.[0-9]{1,3}){3}\b' security_log`

```
(gopika@kali)~/linux/lab/lab4
$ grep -Eo '\b10(\.[0-9]{1,3}){3}\b' security_log | head -5
10.10.5.23
10.20.30.40
10.1.2.3
10.1.2.3
10.1.2.3
```

Extract all addresses belonging to the 192.168.x.x network.

`grep -Eo '\b192\.168(\.[0-9]{1,3}){2}\b' security_log`

```
(gopika@kali)~/linux/lab/lab4
$ grep -Eo '\b192\.168(\.[0-9]{1,3}){2}\b' security_log | head -5
192.168.10.5
192.168.10.21
192.168.10.21
192.168.1.50
192.168.10.5
```

Extract all email addresses.

`grep -Eo '[:alnum:]._%+-]+@[[:alnum:].-]+\.[[:alpha:]]{2,}' security_log`

```
(gopika@kali)~/linux/lab/lab4
$ grep -Eo '[:alnum:]._%+-]+@[[:alnum:].-]+\.[[:alpha:]]{2,}' security_log | head -5
attacker123@phishing-example.com
security.team@example.com
phishing.alert@evilmail.ru
student2025@university.in
(gopika@kali)~/linux/lab/lab4
$ grep -Eo '[:alnum:]._%+-]+@[[:alnum:].-]+\.[[:alpha:]]{2,}' security_log
```

Extract only email addresses whose domain ends in .com.

`grep -Eo '[:alnum:]._%+-]+@[[:alnum:].-]+\.[[:alpha:]]{2,}' security_log`

```
$ grep -Eo '[:alnum:]._%+-]+@[[:alnum:].-]+\.[[:alpha:]]{2,}' security_log
attacker123@phishing-example.com
security.team@example.com
phishing.alert@evilmail.ru
student2025@university.in
(gopika@kali)~/linux/lab/lab4
```

Extract email addresses whose username contains at least one digit.

`grep -Eo '[:alnum:]._%+-]*[0-9][[:alnum:]._%+-]*@[[:alnum:].-]+\.[[:alpha:]]{2,}' security_log`

```
(gopika@kali)~/linux/lab/lab4
$ grep -Eo '[:alnum:]._%+-]*[0-9][[:alnum:]._%+-]*@[[:alnum:].-]+\.[[:alpha:]]{2,}' security_log
attacker123@phishing-example.com
security.team@example.com
(gopika@kali)~/linux/lab/lab4
$ grep -Eo '[:alnum:]._%+-]*[0-9][[:alnum:]._%+-]*@[[:alnum:].-]+\.[[:alpha:]]{2,}' security_log
attacker123@phishing-example.com
student2025@university.in
```

Extract all URLs beginning with http:// or https://.

`grep -Eo 'https?://[^\s:]+' security_log`

```
(gopika@kali)-[~/linux/lab/lab4]
$ grep -Eo 'https?://[^\s:]*' security_log
https://evilc2.ddns.net/update.php
http://malware-test.example.org/download.php
http://malware-test.example.org/evil.exe
https://phishing-example.com/invoice_2025.pdf
https://target.example.com/login.php
https://www.example.com/index.html
https://shop.example.com/admin.php
http://legacy.example.net/login.php
https://portal.university.in/login.php
https://updates.bad-domain.ru/update.php
https://updates.bad-domain.ru/update.php
http://evil.example.net/dropper.exe
https://files.example.com/upload.php
https://command.bad-domain.in/backdoor.exe
```

Extract only HTTPS URLs.

`grep -Eo 'https://[^\s:]*' security_log`

```
(gopika@kali)-[~/linux/lab/lab4]
$ grep -Eo 'https://[^\s:]*' security_log
https://evilc2.ddns.net/update.php
https://phishing-example.com/invoice_2025.pdf
https://target.example.com/login.php
https://www.example.com/index.html
https://shop.example.com/admin.php
https://portal.university.in/login.php
https://updates.bad-domain.ru/update.php
https://updates.bad-domain.ru/update.php
https://files.example.com/upload.php
https://command.bad-domain.in/backdoor.exe
```

Extract URLs ending with .php.

`grep -Eo 'https?://[^\s:]*\.php\b' security_log`

```
(gopika@kali)-[~/linux/lab/lab4]
$ grep -Eo 'https?://[^\s:]*\.php\b' security_log
https://evilc2.ddns.net/update.php
http://malware-test.example.org/download.php
https://target.example.com/login.php
https://shop.example.com/admin.php
http://legacy.example.net/login.php
https://portal.university.in/login.php
https://updates.bad-domain.ru/update.php
https://updates.bad-domain.ru/update.php
https://files.example.com/upload.php
```

Extract all CVE identifiers in the form CVE-YYYY-NNNN.

`grep -Eo 'CVE-[0-9]{4}-[0-9]{4,}' security_log`

```
(gopika@kali)-[~/linux/lab/lab4]
$ grep -Eo 'CVE-[0-9]{4}-[0-9]{4,}' security_log
CVE-2025-12345
CVE-2024-3094
CVE-2025-987654
CVE-2023-44487
CVE-2025-54321
CVE-2025-54321
```

Extract only CVEs reported for the year 2025.

`grep -Eo 'CVE-2025-[0-9]{4,}' security_log`

```
(gopika@kali)-[~/linux/lab/lab4]
$ grep -Eo 'CVE-[0-9]{4}-[0-9]{4,}' security_log
CVE-2025-12345
CVE-2024-3094
CVE-2025-987654
CVE-2023-44487
CVE-2025-54321
CVE-2025-54321
```

Extract all 32-character hexadecimal MD5 hashes.

`grep -Eo '\b[0-9A-Fa-f]{32}\b' security_log`

```
(gopika@kali)-[~/linux/lab/lab4]
$ grep -Eo '\b[0-9A-Fa-f]{32}\b' security_log
44d88612fea8a8f36de82e1278abb02f
e2fc714c4727ee9395f324cd2e7f331f
5d41402abc4b2a76b9719d911017c592
```

Extract all 40-character hexadecimal SHA-1 hashes.

`grep -Eo '\b[0-9A-Fa-f]{40}\b' security_log`

Extract all 64-character hexadecimal SHA-256 hashes.

`grep -Eo '\b[0-9A-Fa-f]{64}\b' security_log`

Extract all timestamps having the format YYYY-MM-DD HH:MM:SS.

`grep -Eo '[0-9]{4}-[0-9]{2}-[0-9]{2} [0-9]{2}:[0-9]{2}:[0-9]{2}' security_log`

```
(gopika@kali)-[~/linux/lab/lab4]
$ grep -Eo '[0-9]{4}-[0-9]{2}-[0-9]{2} [0-9]{2}:[0-9]{2}:[0-9]{2}' security_log
2025-08-18 08:10:01
2025-08-18 08:11:14
2025-08-18 08:12:03
2025-08-18 08:12:10
2025-08-18 08:12:17
2025-08-18 08:13:02
```

Display all lines beginning with a timestamp.

`grep -E '^([0-9]{4}-[0-9]{2}-[0-9]{2} [0-9]{2}:[0-9]{2}:[0-9]{2})' security_log`

Part C – Anchors and Regex Patterns

Display all lines beginning with the word Suspicious.

`grep -E '^Suspicious' security_log`

Display lines ending with a numeric value.

`grep -E '[0-9]+$' security_log`

Display lines beginning with ERROR or WARNING.

`grep -E '^(ERROR|WARNING)' security_log`

Display lines whose first character is a digit.

```
grep -E '[0-9]' security_log
```

Find lines ending in .exe.

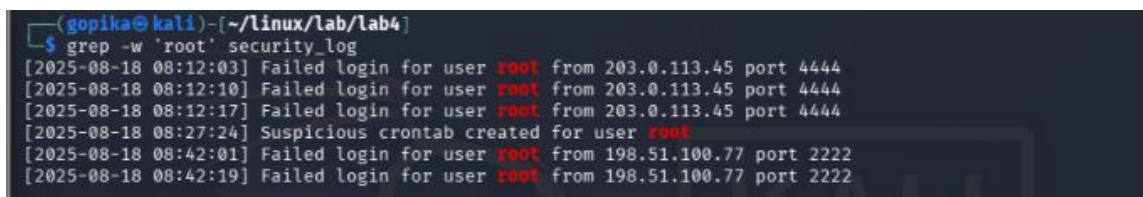
```
grep -E '\.exe$' security_log
```

Find lines containing filenames ending in .exe, .sh, or .pdf.

```
grep -E '[:alnum:]_\.]+\.(exe|sh|pdf)\b' security_log
```

Identify lines containing the exact word root, but not words such as rooted.

```
grep -w 'root' security_log
```



```
(gopika@kali)-[~/linux/lab/lab4]
$ grep -w 'root' security_log
[2025-08-18 08:12:03] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:12:10] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:12:17] Failed login for user root from 203.0.113.45 port 4444
[2025-08-18 08:27:24] Suspicious crontab created for user root
[2025-08-18 08:42:01] Failed login for user root from 198.51.100.77 port 2222
[2025-08-18 08:42:19] Failed login for user root from 198.51.100.77 port 2222
```

Display blank lines in the file.

```
grep '^$' security_log
```

Display all nonblank lines.

```
grep -v '^$' security_log
```

Extract all IPv4 addresses, sort them, and remove duplicates.

```
grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' security_log | sort -u
```

Part D – Linux Filters and Pipelines

Count the number of unique IPv4 addresses present in the log.

```
grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' security_log | sort -u | wc -l
```

Extract all IP addresses and display them in sorted order.

```
grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' security_log | sort
```

Determine which IP addresses appear more than once.

```
grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' security_log | sort | uniq -d
```

Determine the IP address that occurs most frequently.

```
grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' security_log | sort | uniq -c | sort -nr | head -n 1
```

Extract all usernames from authentication entries and display unique usernames.

```
grep -i 'login\|authentication' security_log | grep -Eo 'username[=:]([:alnum:]_\.~)+' | cut -d'=' -f2 | sort -u
```

Count the number of authentication attempts associated with each username.

```
grep -i 'login\|authentication' security_log | grep -Eo 'username[=:]([:alnum:]_\.~)+' | sed -E 's/^username[=:]//' | sort | uniq -c | sort -nr
```

Extract all HTTP status codes and determine how many times each status code occurs.

```
grep -Eo 'status[=:]?[0-9]{3}' security_log | grep -Eo '[0-9]{3}' | sort | uniq -c | sort -nr
```

Display the five most frequently occurring IP addresses.

```
grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' security_log | sort | uniq -c | sort -nr | head -n 5
```

Display the three most frequently occurring usernames.

```
grep -i 'login\|authentication' security_log | grep -Eo 'username[=:]?[[:alnum:]_.-]+' | sed -E 's/^username[=:]?/' | sort | uniq -c | sort -nr | head -n 3
```

Extract all destination ports and display them in numerical order.

```
grep -Eo '(dst[_ -]?port|destination[_ -]?port)[=:]?[0-9]+' security_log | grep -Eo '[0-9]+' | sort -n
```

Remove duplicate port numbers.

```
grep -Eo '(dst[_ -]?port|destination[_ -]?port)[=:]?[0-9]+' security_log | grep -Eo '[0-9]+' | sort -nu
```

Count the number of unique destination ports.

```
grep -Eo '(dst[_ -]?port|destination[_ -]?port)[=:]?[0-9]+' security_log | grep -Eo '[0-9]+' | sort -nu | wc -l
```

Extract all file extensions appearing in the log and determine their frequency.

```
grep -Eo '\.[A-Za-z0-9]{1,10}\b' security_log | tr '[:upper:]' '[:lower:]' | sort | uniq -c | sort -nr
```

Find the five most frequently occurring domains.

```
grep -Eo '([[:alnum:]-]+\.\.){2,}' security_log | tr '[:upper:]' '[:lower:]' | sort | uniq -c | sort -nr | head -n 5
```

Display lines containing both an IP address and the word Failed.

```
grep -E '[0-9]{1,3}\.([0-9]{1,3}){3}' security_log | grep -i 'Failed'
```

Display lines containing both root and Failed.

```
grep -w 'root' security_log | grep -i 'Failed'
```

Display lines containing either download or upload.

```
grep -Ei 'download|upload' security_log
```

Part E – Combining grep Conditions

Display suspicious entries containing either .exe or .sh.

```
grep -Ei 'suspicious' security_log | grep -Ei '\.(exe|sh)\b'
```

Display lines containing a CVE identifier and an IP address.

```
grep -E 'CVE-[0-9]{4}-[0-9]{4},' security_log | grep -E '[0-9]{1,3}\.([0-9]{1,3}){3}'
```

Display lines containing both a filename and a hash value.

```
grep -E '\b([[:alnum:]_.-]+\.(exe|sh|pdf|txt|log))\b' security_log | grep -Ei '[0-9a-f]{32,64}'
```

Display entries containing an IP address followed by a port.

```
grep -E '[0-9]{1,3}\.([0-9]{1,3}){3}:[0-9]{1,5}' security_log
```

Display HTTP requests containing both a URL and status code 404.

```
grep -E 'https?://[^\s:]*' security_log | grep -E '\b404\b'
```

Display authentication failures but exclude entries generated from 127.0.0.1.

```
grep -Ei 'authentication.*fail|failed.*login|login.*fail' security_log | grep -v '127\.\0\.\0\.\1'
```

Display attack-related entries but exclude informational (INFO) messages.

```
grep -Ei 'attack' security_log | grep -v 'INFO'
```

Extract all IPv4 addresses except those beginning with 127.

```
grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' security_log | grep -Ev '^127\.'
```

Display all log entries except successful login events.

```
grep -Eiv 'successful login|login successful' security_log
```

Display all entries that do not contain INFO, DEBUG, or SUCCESS.

```
grep -Ev 'INFO|DEBUG|SUCCESS' security_log
```

Part F – Negative Matching

Find HTTP requests whose status code is not 200.

```
grep -E 'https?://[^\s:]*' security_log | grep -Ev '\b200\b'
```

Display suspicious traffic excluding connections to 192.168.x.x.

```
grep -Ei 'suspicious' security_log | grep -Ev '192\.\168\.[0-9]{1,3}\.[0-9]{1,3}'
```

Extract email addresses except those belonging to the organization's domain.

```
grep -Eo '[[a-zA-Z0-9]_%+]+@[a-zA-Z0-9]_+\.([a-zA-Z]{2,})' security_log | grep -Ev '@organization\.com$'
```

Extract all IPv4 addresses and save them into ips.txt.

```
grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' security_log > ips.txt
```

Extract all email addresses and save them into emails.txt.

```
grep -Eo '[[a-zA-Z0-9]_%+]+@[a-zA-Z0-9]_+\.([a-zA-Z]{2,})' security_log > emails.txt
```

Extract all URLs and save them into urls.txt.

```
grep -Eo 'https?://[^\s:]*' security_log > urls.txt
```

Part G – Redirection

Extract all hashes and save them into hashes.txt.

```
grep -Eo '\b[0-9A-Fa-f]{32}\b|\b[0-9A-Fa-f]{40}\b|\b[0-9A-Fa-f]{64}\b' security_log > hashes.txt
```

Extract failed-login entries and save them into failed_login.txt.

```
grep -Ei 'failed.*login|login.*failed|authentication.*fail' security_log > failed_login.txt
```

Extract suspicious entries and append them to incident_report.txt.

`grep -Ei 'suspicious' security_log >> incident_report.txt`

Store unique IP addresses in unique_ips.txt.

`grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' security_log | sort -u > unique_ips.txt`

Store IP-frequency statistics in ip_frequency.txt.

`grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' security_log | sort | uniq -c | sort -nr > ip_frequency.txt`

Redirect normal output to results.txt and errors to errors.txt.

`some_command > results.txt 2> errors.txt`

Extract domains from Domain accessed entries, lowercase, sort, deduplicate, and save to domains.txt.

`grep -i 'Domain accessed' security_log | grep -Eo '([[:alnum:]]+\.){2,}' | tr '[:upper:]' '[:lower:]' | sort -u > domains.txt`

Identify IP addresses involved in failed-login attempts and save one unique IP per line to failed_login_ips.txt.

`grep -Ei 'failed.*login|login.*failed|authentication.*fail' security_log | grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' | sort -u > failed_login_ips.txt`

Extract all failed-login usernames and save them into failed_users.txt.

`grep -Ei 'failed.*login|login.*failed|authentication.*fail' security_log | grep -Eo 'username[=:]([[:alnum:]]+)' | sed -E 's/^username[=:]//' | sort -u > failed_users.txt`

Extract all domains and save them in lowercase into domains.txt.

`grep -Eo '([[:alnum:]]+\.){2,}' security_log | tr '[:upper:]' '[:lower:]' > domains.txt`

Combining Filters

Extract all MD5 hashes, convert them to uppercase, and save them into md5_hashes.txt.

`grep -Eo '\b[0-9A-Fa-f]{32}\b' security_log | tr '[:lower:]' '[:upper:]' > md5_hashes.txt`

Extract source IP addresses from failed-login records, sort them, remove duplicates, and redirect to failed_ips.txt.

`grep -Ei 'failed.*login|login.*failed|authentication.*fail' security_log | grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' | sort -u > failed_ips.txt`

Create clean_log.txt in which all square brackets are removed from the original log.

`sed 's/\[/]/g; s/\]/]/g' security_log > clean_log.txt`

Create normalized_log.txt by converting the complete log file to lowercase.

`tr '[:upper:]' '[:lower:]' < security_log > normalized_log.txt`

Extract all domain names, convert them to lowercase, sort them, remove duplicates, and save to unique_domains.txt.

`grep -Eo '([[:alnum:]]+\.){2,}' security_log | tr '[:upper:]' '[:lower:]' | sort -u > unique_domains.txt`

Find all failed-login entries and extract only the usernames.

```
grep -Ei 'failed.*login|login.*failed|authentication.*fail' security_log | grep -Eo 'username[=:]([[:alnum:]]_[-])+' | sed -E 's/^username[=:]//'
```

Find all failed-login entries, extract only source IP addresses, and display them in sorted order.

```
grep -Ei 'failed.*login|login.*failed|authentication.*fail' security_log | grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' | sort
```

Extract IP addresses associated with failed logins and determine the unique IPs.

```
grep -Ei 'failed.*login|login.*failed|authentication.*fail' security_log | grep -Eo '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' | sort -u
```

Extract usernames involved in failed logins, convert them to uppercase, and remove duplicate values.

```
grep -Ei 'failed.*login|login.*failed|authentication.*fail' security_log | grep -Eo 'username[=:]([[:alnum:]]_[-])+' | sed -E 's/^username[=:]//' | tr '[:lower:]' '[:upper:]' | sort -u
```

Find all lines containing Domain accessed, extract only domain names, and convert them to lowercase.

```
grep -i 'Domain accessed' security_log | grep -Eo '([[:alnum:]]-)+[[:alpha:]]{2,}' | tr '[:upper:]' '[:lower:]'
```

Find all lines containing MD5=, extract only MD5 hash values, and convert hexadecimal letters to uppercase.

```
grep -E 'MD5=' security_log | grep -Eo '[0-9A-Fa-f]{32}' | tr '[:lower:]' '[:upper:]'
```

Find entries containing User-Agent:, extract the user-agent field, and convert it to lowercase.

```
grep 'User-Agent:' security_log | sed -E 's/.*User-Agent:[[:space:]]*//' | tr '[:upper:]' '[:lower:]'
```

Find all CVE-related lines, extract CVE identifiers, and convert them to lowercase.

```
grep -E 'CVE-[0-9]{4}-[0-9]{4,}' security_log | grep -Eo 'CVE-[0-9]{4}-[0-9]{4,}' | tr '[:upper:]' '[:lower:]'
```

Find all entries containing .exe, extract the relevant filename, and convert it to uppercase.

```
grep -E '\.exe\b' security_log | grep -Eo '([[:alnum:]]_[-])+\.exe\b' | tr '[:lower:]' '[:upper:]'
```

Extract all domains from Domain accessed entries, convert them to uppercase, sort them, and remove duplicates.

```
grep -i 'Domain accessed' security_log | grep -Eo '([[:alnum:]]-)+[[:alpha:]]{2,}' | tr '[:lower:]' '[:upper:]' | sort -u
```

Cheat Sheet

grep 'pattern' file — Search for a pattern in a file.

grep -i — Ignore case.

grep -v — Invert the match; show non-matching lines.

grep -n — Show line numbers.

grep -c — Count matching lines.

grep -l — Show filenames containing a match.

grep -w — Match a complete word.

grep -o — Print only the matching part.

grep -E — Use Extended Regular Expressions.

head -n N — Show first N lines.

tail -n N — Show last N lines.

| — Pipe output of one command into another.

> — Redirect stdout and overwrite/create a file.

>> — Redirect stdout and append to a file.

2> — Redirect error output (stderr) to a file.

< — Use a file as standard input.

sort — Sort lines.

sort -n — Numerical sorting.

sort -r — Reverse sorting.

sort -u — Sort and remove duplicates.

uniq -c — Count consecutive duplicate lines.

uniq -d — Show only duplicated lines.

wc -l — Count lines.

sed 's/a/b/g' — Replace a with b globally in each line.

tr '[:upper:]' '[:lower:]' — Convert uppercase letters to lowercase.

tr '[:lower:]' '[:upper:]' — Convert lowercase letters to uppercase.

^ — Beginning-of-line anchor.

\$ — End-of-line anchor.

. — Any single character in regex.

\. — Literal dot.

* — Zero or more repetitions.

+ — One or more repetitions with grep -E.

? — Zero or one occurrence with grep -E.

{n} — Exactly n repetitions.

{n,m} — Between n and m repetitions.

[0-9] — Any digit.

[A-Za-z] — Any English letter.

[:alnum:] — Letter or digit.

[:space:] — Whitespace character.

[^...] — Any character except those listed.

(A|B) — Group with OR alternatives.

\b — Word boundary in GNU grep regex.

Common Extraction Patterns

IPv4 — \b([0-9]{1,3}\.){3}[0-9]{1,3}\b

Email — [[:alnum:]]_%-]+@[[:alnum:]]-]+\.[[:alpha:]]{2,}

HTTP/HTTPS URL — https?://[^[:space:]]+

CVE — CVE-[0-9]{4}-[0-9]{4}

MD5 — \b[0-9A-Fa-f]{32}\b

SHA-1 — \b[0-9A-Fa-f]{40}\b

SHA-256 — \b[0-9A-Fa-f]{64}\b

Timestamp — [0-9]{4}-[0-9]{2}-[0-9]{2} [0-9]{2}:[0-9]{2}:[0-9]{2}

Filename — [[:alnum:]]_%-]+\.(exe|sh|pdf)\b

Domain — ([[:alnum:]]-]+\.[[:alpha:]]{2,})